# A Collaborative Workflow for Human-AI Research in Pure Mathematics

Hisashi Aoi*

April 2026

**Abstract**

We present computational results on the non-triviality of $\gamma$-deformations of the oscillator Lie superalgebra $B(0,n) = \mathfrak{osp}(1|2n)$: using exact rational arithmetic, we verify non-triviality for $n = 2, \ldots, 5$ (28 cases in total) and obtain explicit non-triviality certificates for each case. These results were achieved through the development of a 4-layer JSON schema-based data pipeline and a structured human-AI collaborative research workflow.

This paper presents this workflow as a reusable workflow for AI-assisted research in pure mathematics, evaluated through one primary case study and one secondary application. The workflow is based on version control with Git and structured documentation, addressing three key challenges: maintaining mathematical rigor, ensuring session continuity across AI interactions, and incrementally implementing theories. Through over 20 issue cycles and more than 100 addressed items, we developed what we call the "Issue-Driven Research Workflow."

The main contributions are: (1) the computational verification of non-triviality of $\gamma$-deformations of $\mathfrak{osp}(1|2n)$ for $n = 2, \ldots, 5$ with explicit non-triviality certificates; (2) design principles for a generic human-AI collaborative workflow for mathematical research, independent of specific AI models; (3) explicit definition of the boundary between tasks that can be delegated to AI and those that cannot; and (4) preliminary evidence of the workflow's transferability to a different research project type (benchmark dataset design for mathematical question formulation).

## 1 Introduction

With the rapid development of large language models (LLM), the utilization of AI in mathematical research has become an increasingly viable option [9, 12]. However, existing approaches have diverged in two major directions. On one hand, some approaches combine formal proof assistants such

*Department of Mathematical Sciences, Ritsumeikan University

as Lean/Coq with LLMs to pursue mechanized guarantees of mathematical correctness [6, 7]. On the other hand, other approaches remain at the level of black-box interactive use, without systematically handling context inheritance between sessions or reproducibility of the research process.

In this paper, we study the non-triviality of $\gamma$-deformations of the oscillator Lie superalgebra $B(0,n) = \mathfrak{osp}(1|2n)$, and present a structured collaborative workflow for human-AI research in pure mathematics that was developed and evaluated through this investigation. The design principles of the proposed workflow are simple. We follow a standard approach for mathematicians—writing theory and proofs in natural language and discussing them with collaborators. For this, we adopt Git [4], a distributed version control system based on branching and merging, for version control and structured documentation for knowledge sharing. These practices are common in software engineering but have not been systematically applied to mathematical research with AI collaboration.

With this approach, we can integrate AI into our research workflow because these knowledge-sharing mechanisms are machine-readable for AI.

The main contributions of this paper are:

1. **Mathematical results**: Computational verification of the non-triviality of all $\gamma$-deformations of $B(0,n) = \mathfrak{osp}(1|2n)$ for $n = 2, \ldots, 5$, together with explicit non-triviality certificates for all 28 cases via exact rational arithmetic.
2. **Workflow design principles**: A set of design principles for a generic human-AI collaborative workflow for mathematical research, independent of specific AI models, based on version control and structured documentation.
3. **Explicit definition of AI delegation boundaries**: We explicitly define the tasks that can be delegated to AI and those that cannot, structurally ensuring safety in human-AI collaboration.
4. **Preliminary evidence of transferability**: As a secondary case study, we report the application of the same workflow to a different research project type (benchmark dataset design for mathematical question formulation), providing preliminary evidence of the workflow's transferability.

The structure of this paper is as follows. §2 describes related work and positions our workflow within it. §3 presents the overall design of the workflow (including the issue-driven workflow extension and the AI delegation boundary policy), §4 discusses the role division between humans and AI, §5 presents methodological insights, §6 describes lessons learned and limitations, and §7 discusses future directions. §8 is the core section of this paper, using the project on verifying the non-triviality of $\gamma$-deformations of the oscillator Lie superalgebra $\mathfrak{osp}(1|2n)$ as the primary case study, detailing how the aforementioned workflow contributed to specific mathematical discoveries. §9 briefly reports a secondary application of the same workflow

to a research project planning task.

## 2 Background and Related Work

Human-AI collaborative research in mathematics can be broadly categorized into three categories from the perspective of technical approaches (Table 1).

### 2.1 Category A: Integration of Formal Proof Systems and LLMs

Attempts to combine formal proof systems like Lean 4 and Coq with LLMs have been rapidly developing in recent years [6, 7].

LeanDojo [6] provides a framework for real-time tactic suggestions and proof completion by LLMs within Lean 4, and DeepSeek-Prover [7] significantly improves theorem proving accuracy in Lean 4 through large-scale synthetic data.

These approaches have achieved significant progress in mechanized proof generation and verification, but they are limited to the scope of formal proof systems and do not systematically address the management of the entire research process (problem setting, strategy design, implementation, documentation) or the continuity of research across multiple AI sessions. The strength of this category lies in the **mechanical guarantee** of mathematical correctness. However, the formalization cost is high, and there is a significant deviation from the informal description styles that mathematicians naturally use (e.g., $\varepsilon$-$\delta$ arguments, diagrams, intuitive reasoning). Moreover, the management of the entire research process (problem setting, strategy formulation, implementation, documentation) is outside their scope.

### 2.2 Category B: Informal human-in-the-loop Workflows

Li et al. [8] propose a human-in-the-loop framework for mathematical research that allows humans to maintain problem settings and assumptions while LLMs rapidly explore proof candidates and counterexamples. This is the most closely related prior work to our workflow, sharing the fundamental principle of "humans maintaining mathematical judgment while AI handles exploration."

Henkel [9] proposes 5 principles for integrating LLMs into the entire research lifecycle as an "augmented mathematician". Our workflow can be positioned as a concrete implementation example of this conceptual framework.

These prior works have contributed to the field, but our workflow offers additional contributions through the combination of: Git for session continuity, structured issue files for requirements management, and reproducible

computational implementation pipelines, enabling research continuity across multiple AI sessions (10 or more in our case study).

### 2.3 Category C: Autonomous Agent Types

Autonomous agent types, represented by MLR-Copilot [10], involve AI agents that consistently handle idea generation, implementation, and execution. As in machine learning research, these approaches have shown considerable success, but in pure mathematics it remains challenging for AI to autonomously guarantee the mathematical validity of proofs [13]. Our workflow adopts a design that intentionally retains human judgment.

### 2.4 Our approach and its novelty

Table 1: Comparison with Related Work

| Approach | Guarantee | Cont. | Pipeline |
|---|---|---|---|
| LeanDojo [6] | Mechanical | None | None |
| DeepSeek-Prover [7] | Mechanical | None | None |
| Li et al. [8] | Human | Not Specified | None |
| Henkel [9] | Principles | Not Specified | Not Specified |
| MLR-Copilot [10] | Autonomous | Within Session | Yes |
| **Our Workflow** | **Human** | **Handover** | **Schemas** |

Compared to the approaches in Categories A (formal proof systems), B (informal human-in-the-loop), and C (autonomous agents), our workflow offers the following novel contributions:

Compared to Category A: While formal proof systems provide mechanical guarantees of mathematical correctness, they have a high formalization cost and deviate from the natural description styles used by mathematicians. Our workflow does not have formal guarantees but achieves accelerated exploration and reproducible implementation through human-AI collaboration while maintaining the natural description style used by mathematicians. The two approaches are not competing but rather complementary methods for different stages and purposes (see §7).

We note that the primary case study in this paper is based on a single mathematical project (see §6), with a second case study (§9) providing preliminary evidence of transferability to research planning tasks. However, the mathematical object treated here—a Lie superalgebra problem with structured coefficients and finite-dimensional verification—is representative of a broader class of computation-heavy projects in representation theory. Accordingly, the JSON schema developed in this study should be regarded as a

reusable design pattern that may be adapted to related objects, rather than as a universal schema.

# 3 Human-AI Collaboration Workflow Framework

In this section, we describe the design principles of a human-AI collaborative workflow applicable to mathematical research projects. Here, we present a framework that is not tied to a single project, but is intended as a reusable workflow pattern for pure mathematical research involving computations. In particular, the "structured data + computational verification" pattern of this workflow appears promising in several areas of mathematics where fieldwork or experiments are not required, such as representation theory (computational verification of character formulas and branching rules), algebraic combinatorics (computational exploration of symmetric groups and combinatorial identities), number theory (Fourier coefficient calculations of modular forms and numerical evaluation of L-functions), and algebraic geometry (ideal calculations via Gröbner bases and verification of intersection numbers). In each case, however, the data structures, verification steps, and task boundaries must be redesigned for the specific project.

## 3.1 Workflow Design Principles

This workflow is based on the following 4 principles:

1. **Pre-planning Principle**: We first present the plan and examine its validity before any implementation, computation, or document creation. Our results must be generated in a verifiable format. These strategies allow us to detect deviations from the intended direction of the AI sessions at an early stage.

2. **Contextual Continuity**: Since LLMs do not have memory across sessions, the continuity of research is maintained only through explicit handover of context. By combining handover documents, Git branch management, and structured documentation, we ensure consistency of research across sessions.

3. **Issue-Driven Workflow**: By managing human instructions, questions, and correction requests as structured files, we eliminate ambiguous instructions given orally or in informal text.

4. **Separation of Exploration and Verification**: We explicitly design an asymmetric division of roles where AI rapidly explores and generates candidates, while humans select and verify mathematically meaningful structures. This separation maximizes the strengths of AI (massive computation and pattern extraction) and humans (mathematical judgment and creative problem setting).

### 3.2 Session Execution Methods

There are multiple execution methods for collaboration with AI, depending on the environment and objectives (Table 2). The choice of method depends on project constraints (security policies, local environment availability, and nature of the work).

Table 2: Comparison of AI Session Execution Methods

| Method | Exec. subject | Access | Main use |
|---|---|---|---|
| Method A | Human (local device) | Read-only | Review, consultation, minor fixes |
| Method B | AI agent + issue workflow | Read/Write | Implementation, computation, research project management |

Figure 1 gives a method-neutral view of the workflow proposed in this paper. Both Method A and Method B share the same structure: human-defined goals, explicit externalization of context, human validation, and Git-based carryover between sessions. They differ only in the execution mode used between context preparation and review.

In any method, the common rules described in §3.3 apply. Regardless of the method used, the **Pre-planning Principle** (§3.1) must be adhered to.

### 3.3 Common Rules and Git Policies

The common rules applicable to all session methods are as follows:

#### Branching Policy

1. When starting a session, create a branch following the naming convention `ai/<YYYY-MM-DD>-<topic>`.
2. As a general rule, each session should have its own dedicated branch.
3. Merging into `main` should be done using `git merge --squash` to keep the history clean.
4. Remove temporary files and debug outputs before merging.

#### Handover Document Policy

- **Latest Notes**: Only the end state of the previous session is retained (past information is moved to the archive).
- **Next Session Instructions**: Instruction files detailing the goals and considerations for the next session are placed.
- **Session Logs**: The content, outcomes, and outstanding tasks of each session are recorded across the project.

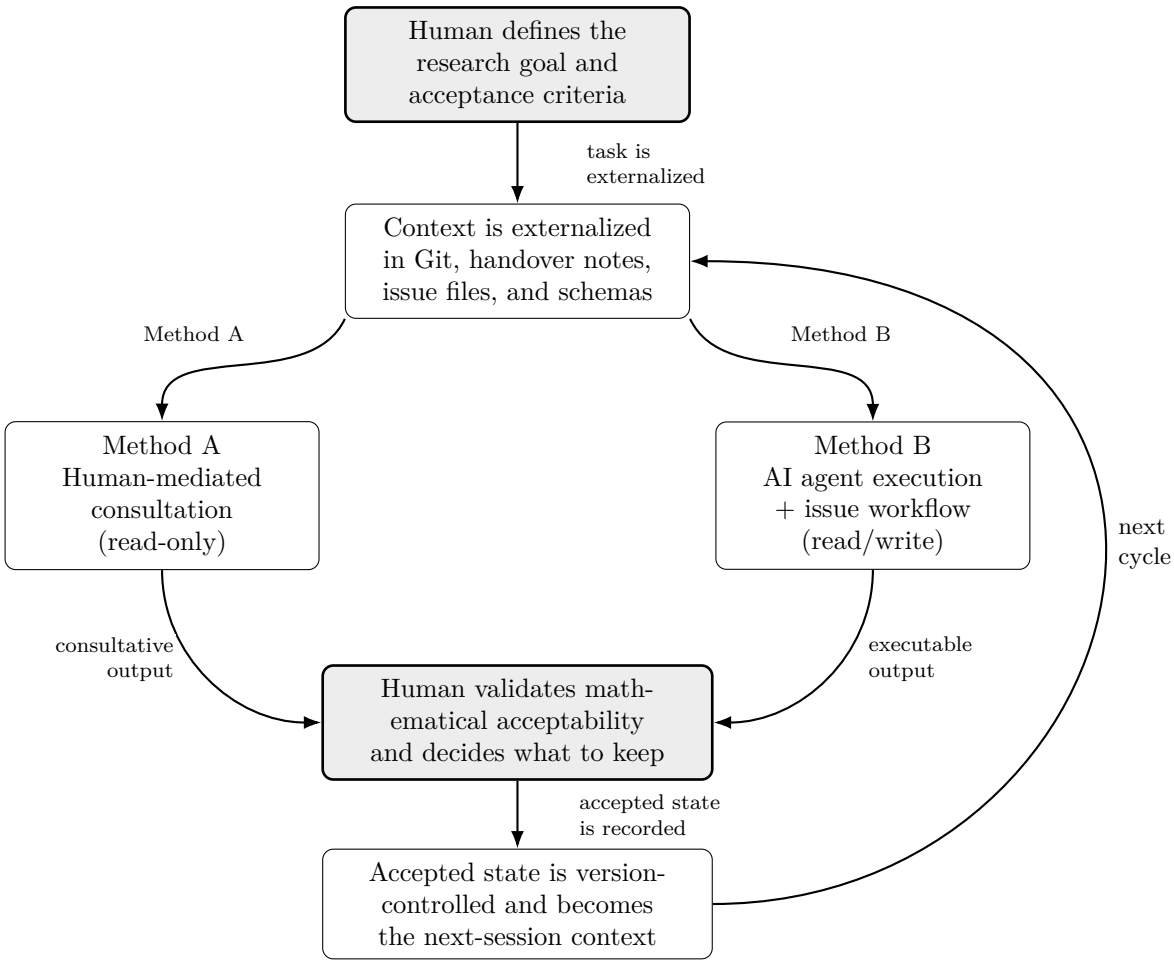


Figure 1: High-level workflow underlying both Method A and Method B. The workflow is common up to context externalization and human validation; the two methods differ only in how AI assistance is executed.

#### Session Checklist

Check the following at the start and end of each session:

**At Start:** Loading handover documents/ Branch verification or creation/ Current status check (`git status`, `git log`)/ Session goal confirmation with human.

**At End:** Check tests passing (if tests exist)/ `git status` confirmation/ Branch push/ Handover document update/ Next session instructions creation/ Temporary file removal.

Method A is the lighter-weight execution mode. It is suited to review, consultation, and minor revisions in environments where direct AI-side execution is not desired. Figure 2 sketches a typical Method A session.

### 3.4 Issue-driven Workflow

The issue-driven workflow is an independent quality management mechanism that functions alongside Methods A and B. Issues (comments, questions, correction requests) created by humans are managed as structured files, determining the priority of AI sessions.

#### Structure of Issue Files

Issue files follow this schema:

- **Header**: Creation date, completion date, status, category, priority
- **Items**: Numbered list with type tags (e.g., "Question", "Instruction", "Correction", "Suggestion", "Confirmation")
- **Responses**: Response field for the assignee (added to the same file)

The status values are as follows:

- `open`: not addressed
- `needs_clarification`: awaiting clarification
- `done`: addressed

Items with status `needs_clarification` are treated as the highest priority.

#### Advantages of Issue-driven Approach

- Questions and answers remain in the same file, making it easy to verify the completeness of responses
- Being file-based, it functions for both interactive and agent-based approaches
- Change history is trackable through version control
- AI response accuracy tends to be higher than verbal instructions (due to reduced ambiguity in structured descriptions)

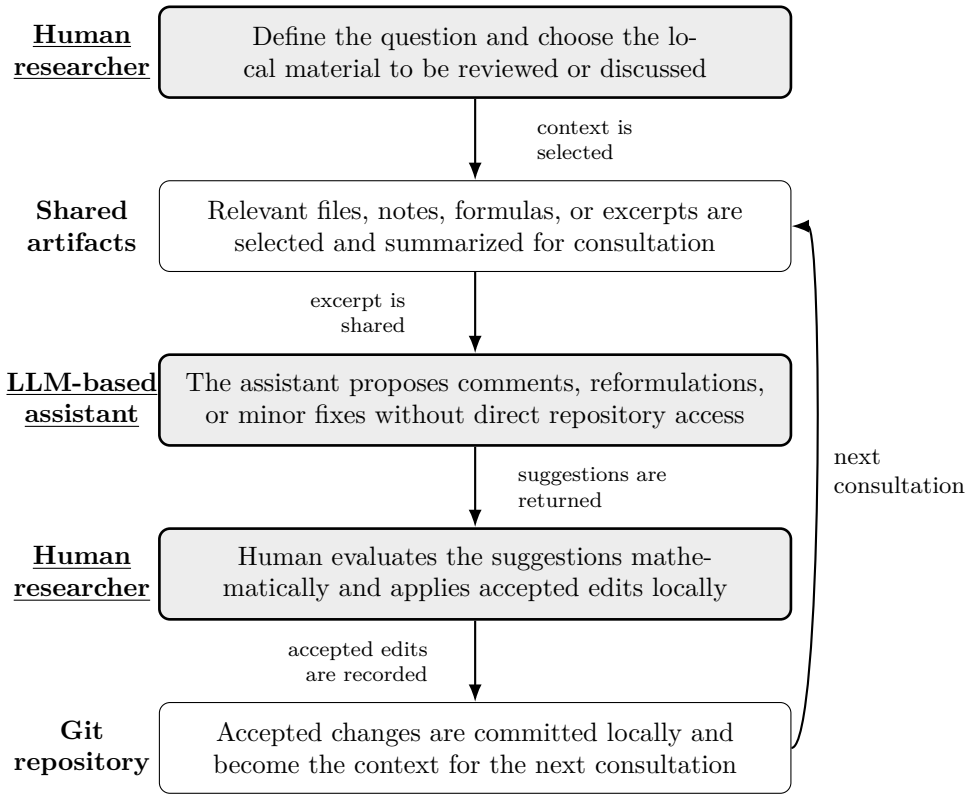


Figure 2: Typical session cycle of Method A. The human researcher keeps the repository local, shares only selected context with the AI assistant, and applies accepted edits manually.

### 3.5 Issue-driven Research Workflow (Method B Extension)

Practical operational experience (12 issue cycles, over 70 addressed items) led to the following extensions of Method B, which integrate AI agent execution with the issue-driven workflow (§3.4).

These extensions are designed to manage continuous research projects with multiple AI sessions, ensuring the continuity of research context and the reproducibility of the research process.

1. **AI Reading Confirmation Protocol**: A mechanism to verify that the AI has correctly read the handover documents at the start of each session, which did not exist previously. In Method B, the AI is required to output the "current branch name, previous results, and current goals" at the start of each session, allowing humans to confirm these details before beginning work.
2. **Explicit Definition of AI Delegation Boundaries**: Explicitly define the tasks that can be delegated to the AI (e.g., computation implementation, testing, LaTeX formatting) and those that cannot (e.g., final judgment on theorem correctness) as project-specific documents (§3.6).
3. **Simplification of Session Management**: Eliminate dual management of Git and handover documents, delegating session log recording to Git's history. Only record research-related judgments (reasons for design decisions, rejected approaches, information that cannot be replaced by Git) separately.

Figure 3 summarizes the core session cycle of Method B as the more operational counterpart to the lighter consultative cycle of Method A (Figure 2). The key point is that Git branches preserve session locality, issue files and handover documents preserve explicit context, and the final mathematical judgment remains with the human researcher. The labels on the left indicate the primary actor or execution context at each stage, while the downward arrows indicate successive handoffs of instructions, generated artifacts, and validation results. In particular, the figure distinguishes the LLM-based assistant from deterministic programs executed by the system.

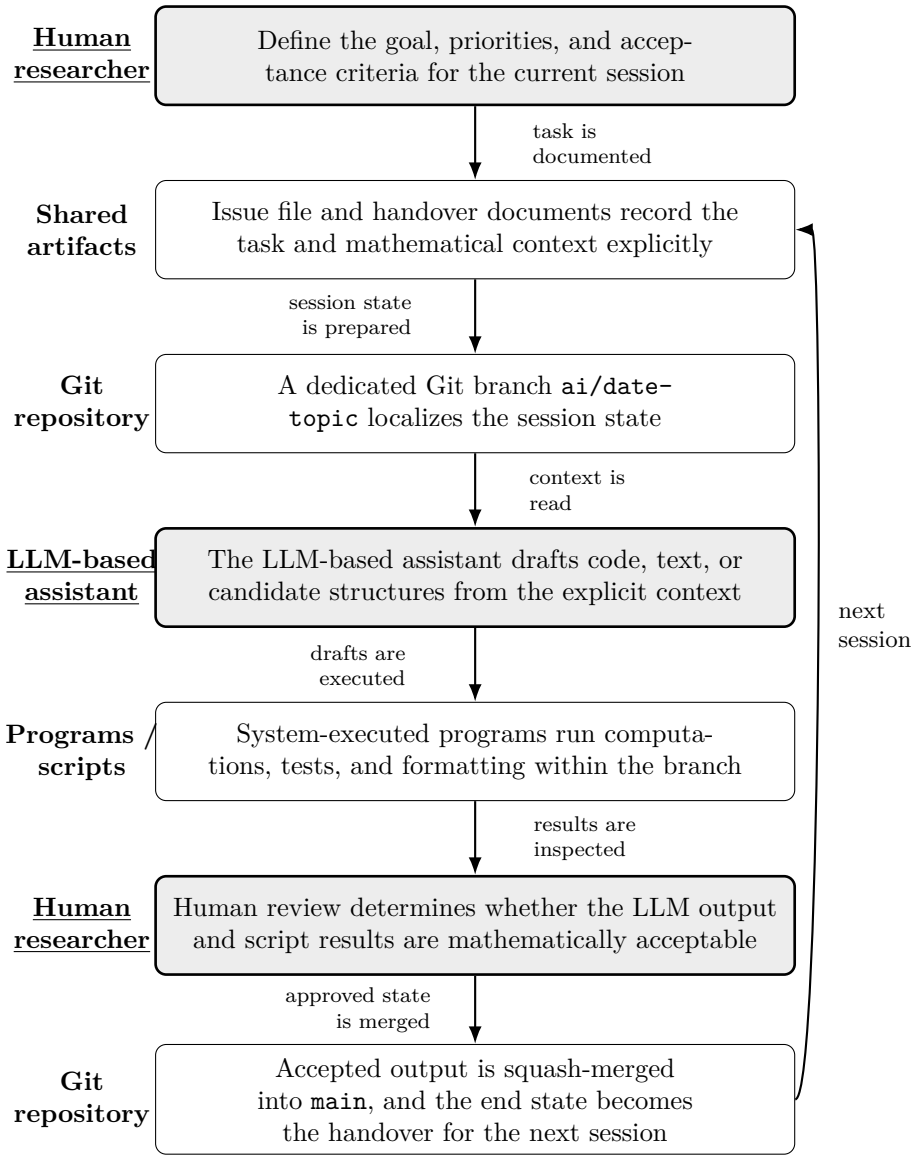


Figure 3: Vertical session cycle of the issue-driven human–AI research workflow. The labels on the left identify the primary actor or execution context at each step, and the downward arrows represent successive handoffs of instructions, generated artifacts, and validation results.

### 3.6 AI Delegation Boundaries Policy

In research involving AI assistants, explicitly defining the boundaries between tasks that can be delegated to the AI and those that require human judgment at the project level is a domain that has not been systematically designed in existing workflow tools.

In this workflow, the project root contains the `ai_trust_policy.md` document that specifies the following:

- **Tasks that can be delegated to the AI**: Computation implementation, test code generation and execution, LaTeX formatting, code refactoring, draft document creation, Git operations
- **Tasks that cannot be delegated to the AI**: Final judgment on theorem correctness, logical validity confirmation of proof steps, evaluation of new mathematical claims, paper submission decisions
- **Trust Conditions**: Pass all tests, understandability of output by humans, consistency with known results, continuation of existing test passes

This delegation boundary document serves as the basis for returning tasks to humans when the AI is asked to make judgments beyond its trust boundaries, thereby structurally ensuring safety in human-AI collaboration.

## 4 Role Division and Implementation Responsibilities

### 4.1 Basic Role Division

The basic principles of role division between humans and AI in this workflow are illustrated in Table 3. This division is not limited to specific projects but serves as a broadly applicable guideline for the mathematical research field involving computational tasks.

○: Primary Responsibility, △: Secondary Responsibility, —: No Involvement.

The core of this division lies in the asymmetry where **mathematical conceptual judgment** (problem setting, proof strategy, final validity judgment) is always retained by humans, while **execution of formalized tasks** (computation, code generation, document drafting) is handled by AI.

### 4.2 Division of Labor in Coding Implementation

The feature of this workflow is that it enables complementary division of labor between an AI proficient in coding and a mathematician who is not good at coding but has deep insights into the theory.

The conditions for this division to function effectively are as follows:

Table 3: Basic Role Division between Humans and AI

| Task | Human | AI |
|---|---|---|
| Problem Setting | ○ | — |
| Proof Strategy Design | ○ | △ |
| Data Structure | ○ | △ |
| Algorithm Direction | ○ | △ |
| Code Implementation (Formalized Tasks) | △ | ○ |
| Large-Scale Iterative Computation | — | ○ |
| Pattern Extraction from Computational Results | △ | ○ |
| Mathematical Output Validation | ○ | △ |
| Formal Proof Description | ○ | ○ |
| Document Creation (First Draft) | △ | ○ |
| Document Creation (Review & Revision) | ○ | △ |
| Final Judgment on Mathematical Validity | ○ | — |

1. **Output must be in a mathematically interpretable format**: Mathematicians can verify the mathematical correctness of computational results independently of reading the code (e.g., whether the left null space vectors of a matrix satisfy the conditions, or whether structure constants satisfy algebraic relations). This verifiability mitigates trust issues in AI-generated code for non-programmers [11].

2. **Tasks must be well-formulated**: AI's accuracy is high only for "preset algorithm implementations." For non-routine or creative implementation tasks, AI's contribution tends to be unstable. Explicitly specifying the direction of algorithms by mathematicians leads to significant improvements in implementation efficiency (based on observations from this project, and the empirical validation as a general proposition remains a future challenge).

3. **Regression detection via testing must function**: The correctness of AI-generated code is independently verified through tests on mathematically meaningful input-output pairs. The mandatory testing policy enables early detection of implicit specification changes across sessions.

### 4.3 Process of "Verbalizing Mathematical Theories"

The genuinely novel contribution of this workflow is the process of **gradual verbalization of unformalized mathematical theories** through itera-

tive dialogue with AI.

In the past, implementing "strict but not algorithmically formalized theories" (e.g., outlines of proof strategies, intuitive classifications of families, geometric meanings of coefficient formulas) into computational form required either mathematicians to possess coding skills or close collaboration with skilled programmers.

In this workflow, this process is realized through the following cycle:

1. Mathematicians describe proof strategies and structural intuitions in natural language and symbols (documented in issue files and handover documents)
2. AI interprets the descriptions and proposes implementation candidates
3. Mathematicians verify the mathematical outputs and refine the direction
4. The cycle of 1–3 is repeated until the theory is gradually implemented

This process is distinct from existing prompt engineering research [12] that focuses on individual prompts, in that it targets the **entire progressive implementation process of theories across multiple sessions**.

### 4.4 Independent of AI Model

The important characteristic of this workflow is the independence from a specific AI model. The handover documents, issue files, and Git branch management are all in standard text formats, making them readable by any LLM-based AI tool.

This property enables the following:

- Easy replacement of AI tools during the research process
- Independent reference of the same handover document by multiple AI models, allowing humans to compare and verify the results generated by different AI models for the same input (e.g., comparing the outputs of GPT-4 and Gemini Pro for the same handover document to identify discrepancies and validate results)
- Low migration cost to future new AI models, as the workflow is not tied to specific features of a particular model

We note that since each workflow artifact is version-controlled with Git, we can also track the history of changes in the workflow itself, which supports the transparency and reproducibility of the research methodology.

The template of this workflow is publicly available in the repository [1], which includes branch policies, handover document templates, and issue file schemas to facilitate application to other projects.

# 5 Insights on Methodology

## 5.1 Separation of Exploration and Verification

The most effective division of labor observed through this workflow is the clear separation of **AI-driven rapid candidate exploration** and **human-driven mathematical selection and verification**.

The conditions under which this separation is effective are as follows:

- The search space is combinatorially large, making exhaustive search computationally expensive
- The judgment of the mathematical value of search results requires domain knowledge
- Candidate generation (exploration) and correctness verification (validation) can be designed as independent steps

AI excels at generating candidates and numerically verifying conditions, but its ability to judge the mathematical significance (why certain structures emerge, whether they are generalizable) of the obtained results is currently limited. Humans take on this judgment role, providing mathematical meaning and direction to the AI's exploration results.

## 5.2 From Pattern Recognition to Theorem

The typical discovery cycle in this workflow follows the structure below:

1. **AI-driven computational exploration**: Execute computations for multiple parameter values and collect numerical data
2. **AI-driven pattern extraction**: Inductively observe common structures and invariants in the data and present them as conjectures
3. **Human-driven mathematical formalization**: Scrutinize the presented conjectures and formalize them as appropriate generalizations and rigorous propositions
4. **Human + AI-driven proof**: Have the AI create drafts of expansion calculations, and have humans verify and revise the mathematical validity of the proof steps
5. **AI-driven computational verification**: Independently verify the proven propositions with specific parameter values to ensure the correctness of the implementation and the proof

This cycle structures the process by which mathematical discoveries progress from computational evidence to theorems. AI excels at collecting computational evidence and extracting patterns, while humans contribute mathematical insight and rigor.

## 5.3 Schema Stability and Productivity

The stability of data structures (schemas) and the productivity of AI show a strong correlation. During stages where schemas are frequently changed,

data compatibility between AI sessions is lost, and the cost of modifications increases.

From this, the following design principles are recommended:

- It is important to carefully design schemas in the initial stages of research and stabilize them as early as possible
- Breaking changes require explicit version number updates
- Backward compatibility should be a principle, prioritizing field additions over deletions

Once schemas are stabilized, AI can reference and generate consistent data across sessions, leading to improved implementation speed and accuracy (quantitative evaluations in this paper are primarily based on structural complexity and modification history, and are indirect indicators rather than direct time measurements).

### 5.4 Evaluation of Exact Arithmetic in Computational Verification

In computational verification, the choice between floating-point arithmetic and exact arithmetic requires careful consideration as a design decision in the early stages of a project.

In the context of computational verification in pure mathematical research, the adoption of exact arithmetic (rational arithmetic and symbolic computation) is recommended:

- Small non-zero components can be accurately detected (there is a risk of incorrect detection with floating-point arithmetic due to rounding errors)
- Numerical-precision-related failure modes in verification scripts can be eliminated, although the logical correctness of AI-generated scripts still requires tests and human review
- The reproducibility of results is substantially less sensitive to the execution environment (OS, library version) than floating-point approaches, since exact arithmetic eliminates rounding differences. (Verified on macOS 14 / Python 3.12 and Ubuntu 22.04 / Python 3.11.)

This decision is difficult to change after implementation, so it is important to finalize it during the early stages of the project.

## 6 Lessons and Limitations

### 6.1 Lessons Learned

1. **Essential Role of Structured Handover**: Since LLMs do not retain memory across sessions, the quality of handover documents directly determines the continuity of the project. The three-layer structure of latest notes, archives, and next-session instructions minimizes

context loss.

2. **Issue-Driven Workflow Maintains Quality**: Structured issue files reduce errors caused by ambiguous verbal instructions. Categorization by type (questions, instructions, revisions) enhances the accuracy of AI responses.

3. **Schema Stability Determines AI Productivity**: The period before schema stabilization sees low AI productivity, while productivity improves after stabilization (§5.3).

4. **Usefulness of Verification-Only Mode**: By explicitly defining a mode where AI only reads information, the cycle of "first accurately grasping the current state, then making changes" can be established. This is particularly effective in bug fixes, refactoring, and proof verification.

5. **Explicit Definition of AI Delegation Boundaries Enhances Reliability**: By explicitly defining the tasks that can and cannot be delegated to AI in project-specific documents, the risk of AI making inappropriate decisions is structurally reduced. In particular, in mathematical research, it is essential not to entrust the final judgment of the correctness of theorems to AI, and the formalization of this boundary serves as the basis for quality assurance.

6. **Improving Implementation Efficiency through Explicit Directional Guidance**: Based on the experience of this project, when mathematicians explicitly guide the direction of algorithms, AI's implementation trial-and-error decreases, leading to improved efficiency. This is an observation based on a single case, but may represent a general trend in mathematical research.

### 6.2 Limitations of the Proposed Workflow

The proposed workflow has the following fundamental limitations. Honestly documenting these limitations is important for accurately assessing the applicability of the workflow.

1. **Limited Case Studies**: The primary evidence for this workflow comes from a single pure mathematics project (§8). A secondary application to research planning (§9) provides preliminary evidence of transferability, but both originate from the same research group. Broader generalizability claims across different teams, institutions, and research domains remain an issue for future verification.

2. **Lack of Formal Guarantees**: The proposed workflow does not mechanically guarantee the formal correctness of mathematical proofs.

The final judgment of mathematical validity is left to humans, making it unsuitable for application areas requiring formal guarantees (e.g., cryptography, verified software).

3. **Limitations of AI Contributions in Highly Creative Problem Formulation**: In the highly creative stage of problem formulation, AI's contributions are currently limited. The proposed workflow is most effective in the "implementation, computation, and verification" stages after the problem is sufficiently formalized.

4. **Dependency on Verifiability of Mathematical Outputs**: The complementary division of labor described in §4.2 only functions when AI outputs are in a mathematically interpretable format. When outputs are difficult to interpret (e.g., numerical solutions of large matrices), independent verification by mathematicians becomes challenging.

5. **Authorship Issues**: The attribution of proof drafts, code, and document drafts generated by AI is currently under development within the mathematical community. The position of this workflow is based on the principle that the final judgment of mathematical validity is held by humans, and this issue should be addressed through community discussions.

6. **Uncertainty in Applicability**: The mathematical object in this case study ($\mathfrak{osp}(1|2n)$'s cohomology equations) has the property that its structure constants can be described by rational numbers, and its proof validity can be verified within the scope of finite-dimensional linear algebra. Whether this workflow is effective for problems that essentially use inductive limit constructions or topological methods remains unverified.

7. **Lack of Quantitative Evaluation of AI Model Dependence**: While qualitative discussions on model independence were conducted in §4.4, quantitative benchmark comparisons regarding the differences in output quality between different models have not been implemented. Future verification tasks include quantitative comparisons of task completion rates and error rates across multiple models.

# 7 Future Work

## 7.1 Parallelization of the Workflow

Since this workflow is based on distributed management of Git, it can be naturally extended to scenarios where multiple human researchers verify different approaches in parallel. Specifically, a form where each researcher collaborates with AI on an independent branch and humans make merge

decisions on mathematically promising results can be considered. This form has a clear distinction from existing multi-agent research where only AI agents handle parallel work [14], in that **humans are responsible for the mathematical verification of each branch**. Git's conflict resolution and merge strategies are expected to function effectively in integrating multiple proof approaches.

### 7.2 Connection with Formal Proof Systems

One of the long-term directions of this workflow is its connection with formal proof systems. The structured proof descriptions generated by this workflow (LaTeX + issue files) are more staged and structured than ordinary mathematical papers as input for informal-to-formal transformation (autoformalization), and the connection with LLM-based Lean 4 formal proof generation [15] is a future consideration.

A candidate for a concrete contribution is the Lean 4 mathematical library mathlib [16]. Currently, the basic structures of Lie algebras (Lie subalgebras, Lie modules, Lie homomorphisms) are formalized in mathlib, but the theory of Lie superalgebras is, as of early 2026, not yet developed. The formalization of $\mathfrak{osp}(1|2n)$, which is the subject of this project, requires $\mathbb{Z}/2\mathbb{Z}$-graded algebra structures, antisymmetry of super brackets and super Jacobi identities, and cohomology theory of deformations, and can be positioned as a natural extension of mathlib's Lie algebra library.

However, we note that the effectiveness of this connection is an empirical question that requires verification. Also, the mechanical guarantees obtained through formalization and the informal flexibility of this workflow are complementary (see §2), and their combination is a promising long-term direction.

### 7.3 Adapting to the Evolution of AI Models

Our workflow is designed to be independent of specific AI models (see §4.4), resulting in low migration costs for the emergence of new AI models. With the expected improvement in LLM capabilities in the future (especially in long-context understanding and mathematical reasoning), it is anticipated that the contribution area of AI in this workflow will expand.

On the other hand, even with the improvement of AI capabilities, we believe that the core roles of **problem setting, proof strategy design, and final judgment of mathematical validity** will continue to be retained by humans, which is essential for maintaining mathematical rigor.

# 8 Case Study: Triviality of Deformations of the Oscillator Lie Superalgebra

This section presents a concrete example of applying the workflow described in §3–§6, namely the project on verifying the non-triviality of $\gamma$-deformations of the oscillator Lie superalgebra $B(0,n) = \mathfrak{osp}(1|2n)$ [5]. The code and data for this project are available in the repository [2], which includes the 4-layer JSON schemas, implementation code, and a sample Jupyter notebook demonstrating verification.

## 8.1 Mathematical Background

Let $\mathfrak{g} = B(0,n) = \mathfrak{osp}(1|2n)$. This Lie superalgebra has Cartan elements $H_1, \ldots, H_n$, root generators $E_\alpha$, an auxiliary fermion $a_0$ ($p(a_0) = 1$, $a_0^2 = \frac{1}{2}$), and boson pairs $b_j^\pm$ ($j = 1, \ldots, n$, $[b_j^+, b_k^-] = \delta_{jk}$) [17]. Consider the odd central extension $\tilde{\mathfrak{g}} = \mathfrak{g} \oplus \mathbb{R}\kappa$, where $\kappa$ is an odd central element. Bakalov–Sullivan [17] introduced a deformed bracket with a parameter matrix $\mathrm{gb} = (\mathrm{gb}_{a_0, b_j^\pm})_{j=1,\ldots,n}$ on this extension:

$$[X, Y]_L = [X, Y]_0 + \kappa \cdot \gamma(X, Y)$$

where $[X, Y]_0$ is the undeformed bracket, and $\gamma \in C^2(\mathfrak{g}, \mathfrak{g})$ is a 2-cocycle linear in gb. Here $\delta\colon C^1(\mathfrak{g}, \mathfrak{g}) \to C^2(\mathfrak{g}, \mathfrak{g})$ denotes the coboundary operator; triviality of the deformation means $\gamma = \delta f$ for some odd $f \in C^1(\mathfrak{g}, \mathfrak{g})$.

The main goal of this project is to determine whether $\gamma$ is a coboundary when $\mathrm{gb} \neq 0$, i.e., whether the deformation is trivial or non-trivial. The definition of a coboundary is as follows: $\gamma$ is a coboundary if there exists an odd linear map $f : \mathfrak{g} \to \mathfrak{g}$ ($p(f) = 1$) such that for any homogeneous elements $a, b \in \mathfrak{g}$,

$$(\delta f)(a, b) = (-1)^{p(a)}[a, f(b)] - (-1)^{(p(a)+1)p(b)}[b, f(a)] - f([a, b]) = \gamma(a, b). \tag{1}$$

(for general elements, the coboundary condition is given by the bilinear extension of this formula).

The proof of non-triviality is carried out by demonstrating that the system of linear equations $Ax = \ell$ is inconsistent, i.e., $\ell \notin \mathrm{Im}(A)$, using a left null space vector $c$ (a certificate):

$$c^\top A = 0 \quad \text{and} \quad c^\top \ell \neq 0. \tag{2}$$

## 8.2 Data Pipeline Based on 4-Layer JSON Schema

In this project, the computation pipeline from structure constants to triviality determination is managed using a 4-layer JSON schema (Table 4). This design ensures that data at each stage can be independently verified, thereby

guaranteeing the reproducibility of the entire pipeline and the compatibility between different AI sessions.

Table 4: 4-Layer JSON Schema Overview

| Schema | File Name | Content |
|---|---|---|
| 1 | `structure.json` | Lie superalgebra structure (basis, parity, oscillator relations) |
| 2 | `gamma_sym.json` | Symbolic gb variables in $\gamma$ components |
| 3 | `gb_flat.json` | Numerical gb values in $\gamma$, complete brackets $a + b\kappa$ |
| 4 | `cob_<tag>.json` | $f$ map $\to \delta f$ coboundary components |

Prefix `B_m_n_` and some file names are abbreviated for readability.

**Forward Flow (**gb **Parameters** $\to \gamma$**):**

$$\text{Schema 1} \longrightarrow \text{Schema 2 (Symbolic } \texttt{gb}\text{)} \longrightarrow \text{Schema 3 (Numerical } \texttt{gb}\text{)}$$

**Backward Flow (**$f$ **Map** $\to \delta f$**):**

$$\text{Schema 1} \longrightarrow \text{Schema 4 } (f \to \delta f)$$

The triviality of the deformation is determined by checking whether $\gamma$ (Schema 3) can be expressed in the form of $\delta f$ (Schema 4), which is the coboundary condition. The non-triviality proof using the certificate method verifies condition (2): the coboundary matrix $A_\mu$ is derived from the bracket structure in Schema 1, and the deformation vector $\ell_\mu$ from the gb values in Schema 3. Verification is performed in Python with exact rational arithmetic via `fractions.Fraction`.

We emphasize that this 4-layer structure directly contributed to the discovery of the certificate. By describing the structure constants precisely in Schema 1 and keeping $\gamma$ as symbolic gb variables in Schema 2, AI was able to analyze the structure of the boundary matrix $A_\mu$ and the deformation matrix $L_\mu$ for each weight sector, leading to the discovery of patterns in the left null space vectors. In particular, the process of inductively conjecturing the $n$-invariant unified construction of Families I–III from the computational results for $n = 2, 3$, and arriving at an algebraic proof for general $n$, would have been difficult without the structured data provided by the schemas.

### Structural Fields of the Schemas

Each schema layer contains numerous structural fields that require mathematically precise specifications, excluding metadata and explanatory text. Table 5 shows the aggregated results of these fields categorized by type.

Table 5: Structural Fields of the Schemas

| Type | S1 | S2 | S3 | S4 | Total |
|---|---|---|---|---|---|
| Algebraic structure | 14 | 5 | 4 | 3 | 26 |
| Ordering | 7 | 2 | 1 | 2 | 12 |
| Relations | 9 | 3 | 0 | 1 | 13 |
| Deformation | 0 | 7 | 5 | 11 | 23 |
| **Total** | **30** | **17** | **10** | **17** | **74** |

Representative fields for each category, together with the mathematical judgments required to specify them, are as follows:

- **Algebraic Structure** (26 fields): These fields include basis names, parity assignments, and oscillator-generator labels. Their specification requires complete root-system enumeration and explicit adherence to Frappat conventions [18], because inconsistencies at this level immediately propagate to later schema layers.
- **Ordering & Indices** (12 fields): These fields determine the PBW ordering, oscillator expansion coefficients, and canonical pairs. Defining them requires mathematical choices about non-commutative normal ordering in the universal enveloping superalgebra, and these choices directly affect whether data generated in different sessions remains compatible.
- **Relations & Constraints** (13 fields): These fields encode structure constants, commutation and anticommutation relations, and metric or symplectic matrices. They must satisfy the super Jacobi identity and mutual consistency conditions, which makes computational verification essential before downstream use.
- **Deformation-specific** (23 fields): These fields specify the inhomogeneous matrix, $\gamma$ components, $f$-map components, and evaluated brackets. Their definition requires both the coboundary computation $\delta f$ and mathematically justified parameter choices, because small specification errors change the certificate system itself.

These 74 fields therefore require explicit Lie-superalgebra knowledge, computational-algebra constraints, and cross-referencing with peer-reviewed literature; they are not fields that can be specified reliably from generic pattern completion alone.

### Schema Version Management and Data Compatibility

Schema versions have been iteratively revised from v1 to v5. Important milestones are as follows:

- **v1–v3**: Exploratory phase. PBW order is not yet determined, and data is often incompatible between AI sessions.
- **v4**: Provisional determination of the Poincaré–Birkhoff–Witt (PBW) order. AI implementation efficiency has started to improve.
- **v5 (stable version)**: PBW order and all schema fields determined. From then on, all AI sessions refer to the same schema, and certificate generation and verification function consistently.

By v5, the schema has been stabilized, and the data compatibility issue between AI sessions caused by schema changes has been resolved, leading to improved productivity. However, it should be noted that the following quantitative evaluation is primarily based on indirect indicators such as structural complexity and modification commit frequency, rather than direct measurements of development time. This is a concrete example of the correlation between schema stability and productivity mentioned in §5.3.

This upward trend can be quantitatively evaluated through the **structural complexity** of the final schema v5. As shown in Table 5, the v5 schema consists of 4 levels and 74 structural fields, with at least 26 top-level fields (Schema 1: 9, Schema 2: 5, Schema 3: 5, Schema 4: 7), each containing a nested structure of up to 4 levels, and including inter-referencing constraints (basis names, PBW order, odd-even consistency across levels). Moreover, to the authors' knowledge, no pre-existing public dataset of the same format exists. To generate the original schema design from scratch using an LLM, it is necessary to specify these structures, constraints, and naming conventions accurately within a single prompt. However, the number of specification items for defining all fields along with their type constraints and inter-level reference consistency is practically over 40 items. Therefore, generating all fields accurately in a single prompt is practically difficult. The frequent schema changes during the exploratory phase (v1–v3) reflect this design complexity. After v5 stabilization, the number of revision commits decreased (over 40

### 8.3 Structural Proof Structure: Family Classification and $n$-invariance

We complete the proof of non-triviality using the certificate method by classifying the certificates $c$ into three Families.

The following lemma is a theoretical result obtained through the use of this project:

**Lemma 8.1** ($j$-th slot invariance)**.** *Certificates belonging to Family I and Family II depend only on the bracket structure within the $j$-th slot, and are independent of $n$. That is, for any $n \geq j+1$, such certificates are valid.*

These patterns were first observed inductively by AI when comparing the computational data for $n = 2, 3, 4, 5$, and were later formally proven by hu-

Table 6: Classification of Certificates by Family

| Family | Representative Structure | $n$ Dependence |
|---|---|---|
| I (Diagonal) | $j$-th slot | $n$-invariant (Lemma 8.1) |
| II (Difference) | adjacent slots | $n$-invariant (Lemma 8.1) |
| III (Cross-term) | cross terms | valid for all $n \geq 2$; special term for $n = 2$ |

mans. The result itself is not particularly special, but the fact that the classification of Families and the conjecture about $n$-invariance were proposed by AI is of great significance. It is a typical example of the collaborative cycle of AI-driven computational discovery and human-driven formalization (§5.2).

About Family III, a formula valid for all $n \geq 2$ was derived, with a special correction term in the $n = 2$ case, and this was confirmed in all cases.

### 8.4 Results of Computational Verification

Finally, the verification of condition (2) for all 28 certificates corresponding to $n = 2, 3, 4, 5$ has been completed (all 28 cases passed).

All data and scripts related to the certificates generated in this project, including JSON data files and verification scripts, are publicly available in the repository [2].

The verification was performed using strict rational number arithmetic via `fractions.Fraction`, completely eliminating numerical errors. The test suite (`pytest tests/ -v`) automatically re-verifies all certificates upon completion of each AI session.

### 8.5 Retrospective on Workflow Application

We retrospectively summarize the main human–AI contributions in applying the workflow to this project:

**Contributions by Humans:** Setting the problem (formulation of the question about the non-triviality of $\gamma$-deformations), proof strategy (adoption of the certificate method), schema design (JSON schemas v1–v5), determination of PBW ordering, Family classification (diagonal, difference, cross-term types), formalization and proof of the theorem (Lemma 8.1), Final judgment on mathematical validity.

**Contributions by AI:** Generation and verification of all 28 certificates ($n = 2, 3, 4, 5$), Computational observation and conjecture formulation of $n$-invariance patterns, Implementation of JSON parser, verification scripts, and schema builder, LaTeX draft generation, continuation documents and session logs creation.

**Project Scale:** AI sessions in total 20 or more, Git commits 100 or more (after squash merge of the main branch), Python scripts 15 files or more, JSON data files 30 files or more.

# 9 Secondary Application: Benchmark Design Project

This section briefly reports a secondary application of the workflow described in §3–§6 to the design of a benchmark dataset called **MathQForm** (Mathematics Question Formulation benchmark) [3]. While the primary case study (§8) demonstrates the workflow in the context of computational proof verification, this application provides preliminary evidence of its transferability to **research project planning**—specifically, the structured design of a dataset, evaluation schema, and data collection protocol through iterative human-AI collaboration.

## 9.1 Project Overview

MathQForm is a benchmark dataset for evaluating the process of transforming informal mathematical notes into structured research questions. The core motivation is that *question formulation*—the ability to extract well-posed, significant research questions from informal sources such as seminar notes—is a critical but underexplored aspect of mathematical research [3].

In the current version, the benchmark is documented bilingually for the B4–M1 level through separate schema, annotation-guide, and rubric documents. The current design already distinguishes the generation paths `human`, `ai`, and `ai_edited`, and uses an evaluation rubric with 4 primary and 2 auxiliary axes. The public project page [3] provides an external documentation layer for this benchmark, so that its motivation and scope are inspectable outside the private design repository used for iterative planning.

## 9.2 Workflow Application

The MathQForm project was conducted entirely under Method B (§3.5), using the same issue-driven workflow, Git branch management, and handover document infrastructure as the primary case study. However, the nature of the tasks differed substantially:

The key observations from applying the workflow to this second project are:

Table 7: Comparison of Workflow Application: Two Case Studies

| Aspect | Case Study 1 (osp) | Case Study 2 (MathQ-Form) |
|---|---|---|
| Domain | Pure mathematics | Research planning |
| Primary Task | Proof verification | Schema & protocol design |
| Data Artifacts | JSON schemas (4-layer) | JSON schemas (6-entity) |
| Verification | Exact arithmetic | Schema validation |
| Issues Addressed | 40+ issues, 100+ items | 26+ issues |
| Managed workspaces | 1 (+ artifacts) | 3 (design + public page + template) |

1. **Schema design as structured dialogue**: The 6-entity schema (Proposition, Item, CandidateQuestion, Annotation, Adjudication, Bundle) was designed through iterative issue cycles, analogous to the 4-layer schema evolution in the primary case study. The schema progressed from an initial 2-entity draft to a 6-entity structure with cross-referencing constraints and evaluation axes (4 primary + 2 auxiliary), demonstrating that the "schema stability → productivity" pattern (§5.3) applies beyond algebraic structure constants.
2. **Multi-repository management**: The project required coordinating across 3 managed workspaces: a private design repository for research planning, a public project page for benchmark documentation and release information [3], and the workflow template repository [1]. The issue-driven workflow provided a unified management mechanism across all three.
3. **Transferability of design principles**: All four design principles (§3.1)—pre-planning, contextual continuity, issue-driven workflow, and separation of exploration and verification—were directly applicable to the research planning context. Notably, the "exploration vs. verification" separation (§5.1) mapped to "AI-driven protocol drafting vs. human evaluation of mathematical appropriateness."

### 9.3 Limitations of This Case Study

This second case study has the following limitations that should be noted:

- The MathQForm project is currently at the pilot stage (schema v0.1.0); the dataset itself has not yet undergone large-scale data collection or external validation.
- Both case studies originate from the same research group, and external replication by independent teams remains an open task.
- The strength of this case study lies in demonstrating *structural transferability* of the workflow (i.e., that the same mechanisms function in

a different task domain), rather than providing a second independent evaluation of research outcomes.

### Data Availability

The data and code supporting the findings of this study are publicly available. The workflow template, handover infrastructure, and issue-driven workflow scripts used throughout this study are available at https://github.com/ritsumei-aoi/ai-research-workflow-template [1]. The computational artifacts for the primary case study—including the 4-layer JSON schemas, computation scripts, and verification results for the triviality of $\gamma$-deformations of $\mathfrak{osp}(1|2n)$—are deposited at https://github.com/ritsumei-aoi/osp-triviality [2].

### About AI Usage and Authorship

This paper reports on the methodology of human–AI collaborative workflows, and by its nature, AI (LLMs) were used for code generation, document drafting, and data verification as part of the research process. The final judgment of mathematical validity and responsibility for the content of the paper lies with the authors. Details of the AI models used and the workflow are as follows:

- AI Models Used: Claude Opus 4.6 (claude-opus-4.6) on GitHub Copilot Chat and GitHub Copilot CLI.
- Usage: Workflows described in §3 with structured handover documents and issue files managed via Git.

## References


[1] H. Aoi, *AI Research Workflow Template*, GitHub repository, https://github.com/ritsumei-aoi/ai-research-workflow-template, 2026.

[2] H. Aoi, *Computational Artifacts for the Triviality of Inhomogeneous Deformations of* $\mathfrak{osp}(1|2n)$, GitHub repository, https://github.com/ritsumei-aoi/osp-triviality, 2026.

[3] H. Aoi, *MathQForm: Mathematics Question Formulation Benchmark*, project website, https://ritsumei-aoi.github.io/mathqform/, 2026.

[4] Git Project, *Git User's Manual*, official documentation, https://git-scm.com/docs/user-manual, accessed April 24, 2026.

[5] H. Aoi, *On the triviality of inhomogeneous deformations of* $\mathfrak{osp}(1|2n)$, arXiv:2604.05252 [math.RT], 2026.

[6] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger and A. Anandkumar, *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, arXiv:2306.15626 [cs.LG], 2023.

[7] H. Xin, D. Guo, Z. Shao, Z. Ren, Q. Zhu, B. Liu, C. Ruan, W. Li and X. Liang, *DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data*, arXiv:2405.14333 [cs.AI], 2024.

[8] C. Li, Z. Lai, D. An, J. Hu and Z. Wen, *Advancing Mathematical Research via Human-AI Interactive Theorem Proving*, arXiv:2512.09443 [cs.HC], 2025.

[9] J. Henkel, *The Mathematician's Assistant: Integrating AI into Research Practice*, arXiv:2508.20236 [math.HO], 2025.

[10] R. Li, T. Patel, Q. Wang and X. Du, *MLR-Copilot: Autonomous Machine Learning Research based on Large Language Models Agents*, arXiv:2408.14033 [cs.AI], 2024.

[11] Y. Virk and D. Liu, *Non-programmers Assessing AI-Generated Code: A Case Study of Business Users Analyzing Data*, arXiv:2508.06484 [cs.HC], 2025.

[12] L. Carbone, *Advancing mathematics research with generative AI*, arXiv:2511.07420 [math.HO], 2025.

[13] S. Frieder, L. Pinchetti, A. Chevalier, R.-R. Griffiths, T. Salvatori, T. Lukasiewicz, P. C. Petersen and J. Berner, *Mathematical Capabilities of ChatGPT*, arXiv:2301.13867 [cs.LG], 2023.

[14] S. Schmidgall and M. Moor, *AgentRxiv: Towards Collaborative Autonomous Research*, arXiv:2503.18102 [cs.AI], 2025.

[15] Z. Wang, B. Yang, C. Li, Y. Zhang, S. Zhou, B. Dong, Z. Wen, A. Wu, B. Clark, and P. Lu, *Translating Informal Proofs into Formal Proofs Using a Chain of States*, arXiv:2512.10317 [cs.LO], 2025.

[16] The mathlib Community, *Mathlib: The Lean 4 Mathematical Library*, https://leanprover-community.github.io/mathlib4_docs/, 2024.

[17] B. Bakalov and M. Sullivan, *Inhomogeneous supersymmetric bilinear forms*, arXiv:1612.09400v2 [math.RT], 2017.

[18] L. Frappat, A. Sciarrino and P. Sorba, *Dictionary on Lie Algebras and Superalgebras*, Academic Press, 2000.